Charles Darwin University

**PRT565 Machine Learning, Artificial Intelligence and Algorithms**

Assessment 3 – Artificial Intelligence Group Presentation (30%)

# Phishing Website Detection

*Comparing machine learning and deep learning models on the UCI Phishing Websites dataset*

## Unit information

| Item | Details |
|---|---|
| Unit | PRT565 Machine Learning, Artificial Intelligence and Algorithms |
| Assessment | Assessment 3 – Group presentation, weighting 30% |
| Unit coordinator / lecturer | [ENTER LECTURER NAME] |
| Submission deadline | 27 September 2026, 22:00 (Sydney time) |
| Group number | [ENTER GROUP NUMBER] |
| Campus | [ENTER CAMPUS] |

## Group members

| Member name | Student ID | Presentation responsibility |
|---|---|---|
| [Presenter A name] | [ENTER ID] | Presenter A – slides 1–9: introduction and motivation, dataset description, EDA, data quality, preprocessing and feature engineering, evaluation design, model overview (≈ 5.5 min) |
| [Presenter B name] | [ENTER ID] | Presenter B – slides 10–18: cross-validation selection, test performance, ROC and error analysis, significance testing, deep learning findings, explainability, conclusion and references (≈ 5.5 min) |

Both members contributed to the code, analysis, report and slides. Total presentation length ≈ 11 minutes (maximum 15 minutes; each member under 6 minutes).

# Recorded presentation and project overview

## Link to recorded presentation

**[PASTE PUBLIC LINK HERE – test it in a private/incognito window: anyone with the link must be able to view/download]**

## Project title

Phishing Website Detection: Comparing Decision Tree, Random Forest, Naive Bayes, Multilayer ANN and Deep Learning Models (LSTM/RNN)

## Brief description

We built and compared eleven supervised classifiers that predict whether a website is phishing or legitimate from 30 engineered indicators in the UCI Phishing Websites dataset (11,055 records). The four required models (Decision Tree, Random Forest, Categorical Naive Bayes and a multilayer ANN) were joined by four additional algorithms (Logistic Regression, k-Nearest Neighbours, SVM and Gradient Boosting) and three deep learning models (a three-layer ANN and bidirectional LSTM and simple RNN networks in PyTorch). All models were tuned on identical group-safe cross-validation folds, the winner was chosen before testing, and every model was evaluated once on the same 2,210 held-out records using accuracy, precision, recall, F1 and ROC AUC, supported by bootstrap confidence intervals, McNemar significance tests and permutation feature importance.

## Motivation for selecting the topic

Phishing is one of the highest-volume cyber-attacks: the Anti-Phishing Working Group observed about 3.8 million phishing attacks in 2025, including 853,244 in the fourth quarter alone (Anti-Phishing Working Group, 2026). Blocklists cannot keep pace with new sites, so models that learn from website characteristics are valuable. The problem also has clearly interpretable, asymmetric errors – a missed phishing site can expose credentials, whereas a false alarm blocks a legitimate site – which makes precision, recall and F1 more informative than accuracy alone. The public, labelled, tabular dataset allowed a rigorous comparison of machine learning and deep learning without collecting live malicious websites, and it differs from the Telco dataset used in Assessment 2.

## Main finding

Random Forest achieved the highest development cross-validation F1 (94.58%) and was selected before testing. On the held-out test set it reached 96.38% accuracy, 95.91% phishing F1 (95% CI 94.3–97.3%) and ROC AUC 0.993, and McNemar tests with Holm correction show its error pattern differs significantly from all ten other models. Deep networks (three-layer ANN, LSTM, RNN) did not outperform tree ensembles on these tabular inputs.

# 1 Dataset description and problem definition

The UCI Phishing Websites dataset (Mohammad & McCluskey, 2012) contains 11,055 website records and 30 integer-coded input features, donated in 2015 under a CC BY 4.0 licence. The features were engineered by the dataset authors (Mohammad et al., 2014) and each takes the category −1 (phishing-like), 0 (suspicious) or +1 (legitimate-like). The experiment loads Training Dataset.arff from the original archive and records its SHA-256 hash.

| Feature group | Count | Examples and what they capture |
|---|---|---|
| Address bar | 12 | SSLfinal_State (HTTPS certificate), Prefix_Suffix ("-" in domain), having_Sub_Domain, URL_Length, having_IP_Address |
| Abnormal content | 6 | URL_of_Anchor (links pointing to other domains), Request_URL, Links_in_tags, SFH (where forms submit) |
| HTML / JavaScript | 5 | Iframe, on_mouseover, RightClick, popUpWidnow, Redirect |
| Domain reputation | 7 | age_of_domain, web_traffic, DNSRecord, Page_Rank, Google_Index, Statistical_report |

The original target Result uses −1 for phishing and +1 for legitimate. We mapped phishing to 1 and legitimate to 0, so every precision, recall and F1 value in this report measures phishing detection. The research questions are: (1) which model achieves the strongest development-set phishing F1 under group-safe evaluation; (2) how does it perform on held-out data and is its advantage statistically significant; and (3) do deeper or recurrent neural networks, or a reduced feature set, change the conclusion?

| Audit measure | Observed value |
|---|---|
| Records and input features | 11,055 records; 30 inputs |
| Class counts | 4,898 phishing (44.3%); 6,157 legitimate (55.7%) |
| Missing input values | 0 |
| Repeated complete rows | 5,206 |
| Repeated input rows | 5,270 |
| Unique input patterns | 5,785 |
| Patterns with conflicting labels | 64 |

*Table source: results/audit.json. Repeated-row counts exclude the first occurrence. Identical inputs do not prove identical websites.*

# 2 Dataset analysis

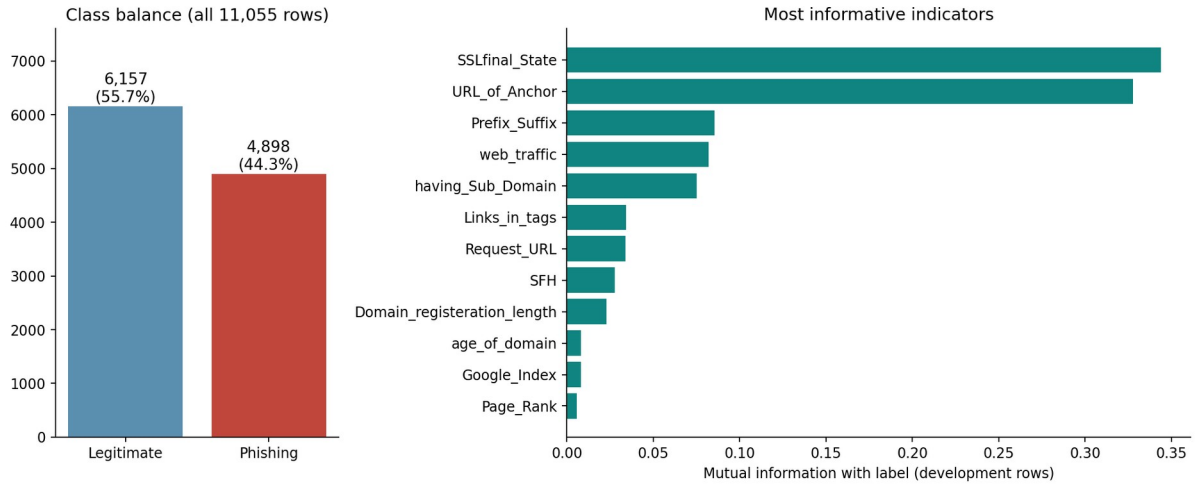



*Figure 1. Class balance and the ten most informative indicators by mutual information (development rows only). Source: results/eda_overview.png.*

The classes are moderately balanced, so accuracy is meaningful, but phishing F1 was used for model selection because it balances missed attacks against false alarms. Mutual information, computed on development rows only to avoid test leakage, shows two dominant indicators: SSLfinal_State (0.34) and URL_of_Anchor (0.33). The next-strongest indicator carries roughly a quarter of that information, so effective models must combine many weaker signals. A Spearman correlation heat map (results/eda_correlation.png) shows moderate correlation among some reputation features, which is relevant to the Naive Bayes independence assumption.

The most important data-quality finding is repetition: only 5,785 of the 11,055 rows have a unique input pattern, and 64 patterns appear with both labels. A random split would place identical patterns in both training and test sets, inflating scores through leakage (Kaufman et al., 2012). This finding directly shaped the evaluation design.

# 3 Preprocessing, feature engineering and evaluation design

## Preprocessing and feature engineering

All observations are retained, including conflicting labels. Exact input patterns are hashed into groups, and a group always stays within one partition. Every transformation lives inside a scikit-learn Pipeline (Pedregosa et al., 2011), so it is fitted on training folds only.

| Step | Applied to | Reason |
|---|---|---|
| One-hot encoding (30 → 90 binary columns, fixed −1/0/+1 vocabulary) | Decision Tree, Random Forest, ANNs, Logistic Regression, k-NN, SVM | Codes are categories; avoids treating −1 < 0 < +1 as a distance |
| Category codes shifted to 0/1/2 | Categorical Naive Bayes, Gradient Boosting (native categorical splits) | These estimators model categories directly |
| Token + learned embedding (feature × code → 90 tokens) | Bidirectional LSTM and simple RNN | Gives recurrent networks a 30-step sequence input |
| Top-10 selection by discrete mutual information, inside each CV fold | Random Forest (extension) | Tests whether a simpler input set preserves performance |
| No scaling or imputation | All | Inputs are bounded (0/1) and complete |

### Evaluation design

The first fold of a seeded five-fold StratifiedGroupKFold (seed 565) defines a 2,210-record test set (979 phishing). The remaining 8,845 development records are divided into three grouped folds (seed 566), identical for every model. GridSearchCV selects settings by mean phishing F1, and the model family with the highest development CV F1 is chosen before test outcomes are calculated. Default 0.5 decision thresholds are used; there is no test-based threshold tuning. Assertions in train.py confirm that no identical input pattern crosses any training–validation or training–test boundary.

On the test set we report accuracy, precision, recall, F1, ROC AUC and confusion matrices; a 95% confidence interval for F1 from 1,000 bootstrap resamples of whole input-pattern groups; and exact McNemar tests (Dietterich, 1998) comparing the selected model with each rival, with Holm correction for multiple comparisons.

## 4 Algorithms and model overview

| Model | Family | Search settings | Selected | CV F1 ± SD |
|---|---|---|---|---|
| Decision Tree | Required | depth 5/none; min leaf 1/5 | none; leaf 5 | 91.69% ± 0.85 |
| **Random Forest** | **Required** | **150 trees; depth 10/none; leaf 1/3** | **none; leaf 1** | **94.58% ± 0.62** |
| Categorical NB | Required | alpha 0.1/1/10 | alpha 1 | 91.76% ± 0.55 |
| Multilayer ANN | Required | hidden (32,16), ReLU, Adam; alpha 0.001/0.01 | alpha 0.01 | 93.62% ± 0.66 |
| Logistic Regression | Extra ML | C 0.1/1/10 | C 10 | 92.99% ± 0.70 |
| k-Nearest Neighbours | Extra ML | k 3/7/15; uniform/distance | k 15, distance | 92.47% ± 0.62 |
| SVM (RBF) | Extra ML | C 1/10 | C 10 | 94.23% ± 0.53 |
| Gradient Boosting | Extra ML | learning rate 0.05/0.1; leaves 15/31 | 0.1; 31 | 94.57% ± 0.49 |
| Deep ANN (3 layers) | Deep learning | hidden (128,64,32); alpha 0.001/0.01 | alpha 0.01 | 94.28% ± 0.96 |
| LSTM (bidirectional) | Deep learning | embed 16, hidden 32×2, dropout 0.2, Adam; early stopping | 23 epochs | 94.09% ± 0.81 |
| Simple RNN (bidirectional) | Deep learning | same as LSTM with plain RNN cell | 14 epochs | 93.63% ± 0.80 |

*Source: train.py, deep_learning_extension.py, results/cv_summary.csv, deep_learning_cv.csv. Searches are small and model-specific; they are not an equal-compute benchmark.*

The Decision Tree learns a sequence of impurity-reducing splits and is the interpretable baseline. Random Forest averages 150 bootstrapped trees with random feature sub-sampling, reducing variance (Breiman, 2001). Categorical Naive Bayes combines per-feature category likelihoods under a conditional-independence assumption, which is restrictive because some indicators are correlated. The required multilayer ANN (MLPClassifier) uses two ReLU hidden layers and Adam; internal early stopping is disabled because its random validation split would break the grouping rule, and no convergence warnings were recorded.

The additional models broaden the comparison: Logistic Regression is a linear benchmark, k-NN classifies by the most similar websites, the RBF SVM learns a maximum-margin non-linear boundary, and histogram Gradient Boosting adds trees sequentially to correct earlier errors. For deep learning we trained a deeper three-layer ANN and, as the brief suggests, bidirectional LSTM (Hochreiter & Schmidhuber, 1997) and simple RNN networks in PyTorch (Paszke et al., 2019). The recurrent models read each website as a sequence of 30 embedded tokens; early stopping on each grouped validation fold chose the epoch count, and the final model was refit on all development rows for the median epoch count. Because epochs were chosen on validation data, the recurrent CV estimates are slightly optimistic.

Random Forest achieved the highest mean CV F1 (94.58%) and was selected as the primary model. Gradient Boosting was effectively tied (94.57%). The forest's training-fold F1 of 98.8% exceeds its validation F1, indicating a normal generalisation gap.

# 5 Model performance evaluation (testing)

| Model | Accuracy | Precision | Recall | F1 [95% CI] | AUC | FN | FP |
|---|---|---|---|---|---|---|---|
| **Random Forest** | **96.38%** | **96.10%** | **95.71%** | **95.91% [94.3–97.3]** | **0.993** | **42** | **38** |
| SVM (RBF) | 95.38% | 96.11% | 93.36% | 94.72% [92.8–96.5] | 0.992 | 65 | 37 |
| Deep ANN (3 layers) | 95.07% | 97.18% | 91.52% | 94.27% [92.3–96.1] | 0.993 | 83 | 26 |
| Gradient Boosting | 94.89% | 94.46% | 93.97% | 94.21% [92.4–96.1] | 0.993 | 59 | 54 |
| Multilayer ANN | 94.39% | 95.05% | 92.13% | 93.57% [91.5–95.6] | 0.991 | 77 | 47 |
| k-Nearest Neighbours | 93.98% | 94.43% | 91.83% | 93.11% [91.2–94.9] | 0.985 | 80 | 53 |
| LSTM (bidirectional) | 93.89% | 94.05% | 92.03% | 93.03% [91.1–94.9] | 0.989 | 78 | 57 |
| Simple RNN (bidirectional) | 93.67% | 93.20% | 92.44% | 92.82% [90.6–94.9] | 0.988 | 74 | 66 |
| Logistic Regression | 93.35% | 93.06% | 91.83% | 92.44% [90.2–94.5] | 0.986 | 80 | 67 |
| Decision Tree | 93.03% | 93.38% | 90.70% | 92.02% [89.7–94.1] | 0.958 | 91 | 63 |
| Categorical NB | 92.94% | 93.09% | 90.81% | 91.93% [89.8–94.0] | 0.981 | 90 | 66 |
| Top 10 Random Forest | 94.16% | 92.25% | 94.79% | 93.50% [91.6–95.3] | 0.985 | 51 | 78 |
| Majority baseline | 55.70% | 0.00% | 0.00% | 0.00% | 0.500 | 979 | 0 |

*Source: results/metrics.csv and deep_learning_metrics.csv. All models use the same 2,210 held-out records; phishing is the positive class. FN = missed phishing, FP = false alarms.*

Random Forest also leads on held-out data with 96.38% accuracy and 95.91% F1, followed by the SVM (94.72%), the deep ANN (94.27%) and Gradient Boosting (94.21%). Among the four required models the ranking is Random Forest, ANN (93.57%), Decision Tree (92.02%) and Categorical NB (91.93%). The majority baseline scores 55.70% accuracy while detecting no phishing, demonstrating why accuracy alone is inadequate.

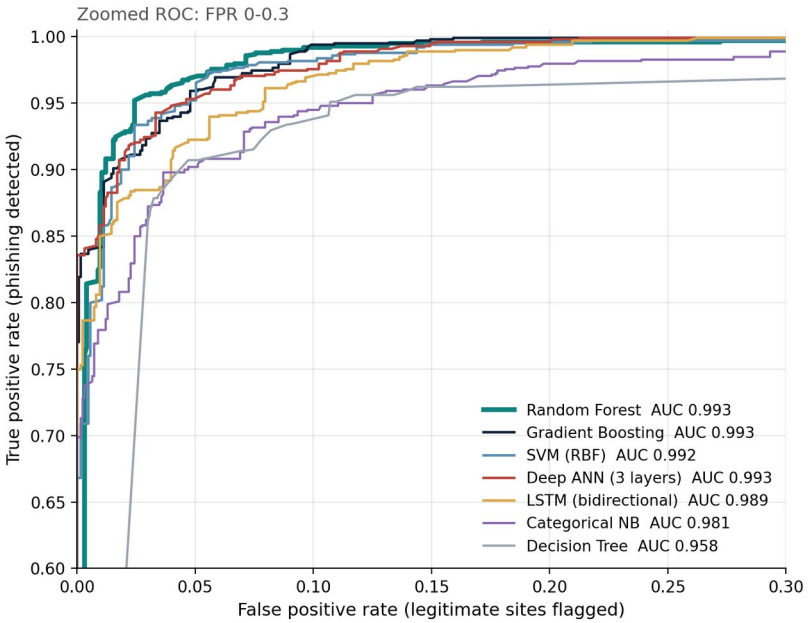



*Figure 2. ROC curves on the test set, zoomed to FPR 0–0.3 (full range: results/roc_curves.png).*

Random Forest and Gradient Boosting share the highest ROC AUC (0.993) and every learned model exceeds 0.95. AUC measures ranking quality across thresholds and does not certify calibrated probabilities; a deployment would choose an operating threshold using the relative cost of missed attacks and false alarms.

## Error analysis

| Actual class | Predicted legitimate | Predicted phishing |
|---|---|---|
| Legitimate | 1,193 true negatives | 38 false positives |
| Phishing | 42 false negatives | 937 true positives |

Precision = 937 / (937 + 38) = 96.10%; recall = 937 / (937 + 42) = 95.71%; accuracy = (1,193 + 937) / 2,210 = 96.38%; F1 = 2 × 937 / (2 × 937 + 38 + 42) = 95.91%. The forest misses 4.29% of phishing records and flags 3.09% of legitimate records. Different models make different trade-offs: the deep ANN produces the fewest false alarms (26) but misses 83 phishing records, twice as many as the forest. For a security application where a missed attack is usually more costly, the forest's balance is preferable.

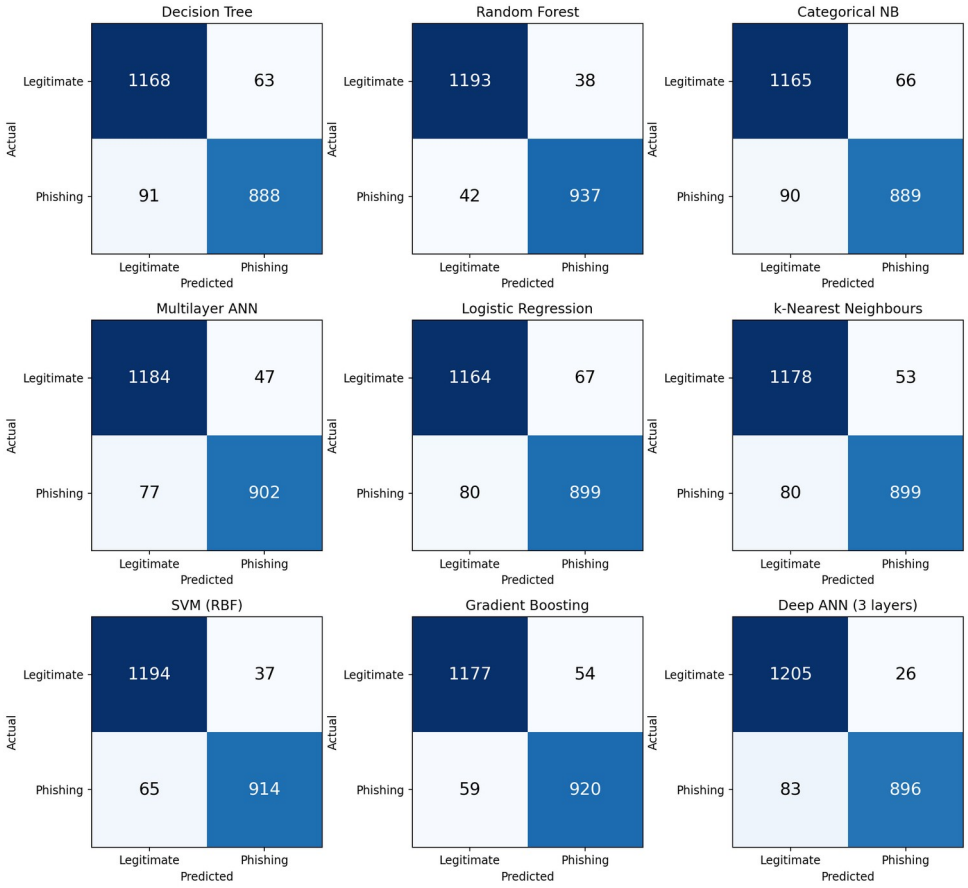



*Figure 3. Confusion matrices for the nine scikit-learn models (results/confusion_matrices.png).*

## 6 Statistical significance

| Compared with Random Forest | RF right, model wrong | Model right, RF wrong | Exact p | Holm-adjusted p |
|---|---|---|---|---|
| SVM (RBF) | 43 | 21 | 0.008 | 0.008 |
| Deep ANN (3 layers) | 59 | 30 | 0.003 | 0.006 |
| Gradient Boosting | 41 | 8 | < 0.001 | < 0.001 |
| Multilayer ANN | 63 | 19 | < 0.001 | < 0.001 |
| LSTM (bidirectional) | 69 | 14 | < 0.001 | < 0.001 |
| Simple RNN (bidirectional) | 75 | 15 | < 0.001 | < 0.001 |
| k-Nearest Neighbours | 68 | 15 | < 0.001 | < 0.001 |
| Logistic Regression | 85 | 18 | < 0.001 | < 0.001 |
| Decision Tree | 81 | 7 | < 0.001 | < 0.001 |
| Categorical NB | 88 | 12 | < 0.001 | < 0.001 |
| Top 10 Random Forest | 70 | 21 | < 0.001 | < 0.001 |

*Source: results/significance_mcnemar.csv and deep_learning_mcnemar.csv; Holm correction across all 11 comparisons.*

McNemar's test considers only records on which two models disagree. The forest wins most disagreements with every rival, and after Holm correction all eleven differences are significant at the 1% level. These results should be read cautiously: the confidence intervals of close rivals (SVM, deep ANN, Gradient Boosting) overlap with the forest's, repeated patterns mean test rows are not fully independent, and the analysis uses a single outer split of one dataset.

## 7 Deep learning findings

Increasing ANN depth from two to three hidden layers improved test F1 from 93.57% to 94.27% and gave the highest precision of any model (97.18%). The bidirectional LSTM (93.03%) and simple RNN (92.82%) performed below the feed-forward networks. Training-loss curves (results/ann_loss_curves.png and dl_loss_curves.png) flatten, so the gap is not simply under-training. The inputs are 30 categorical indicators with no natural order, so recurrent models have no temporal structure to exploit, whereas trees capture threshold-like interactions directly. This is consistent with evidence that tree ensembles frequently outperform deep networks on medium-sized tabular data (Grinsztajn et al., 2022).

# 8 Explainability and feature selection

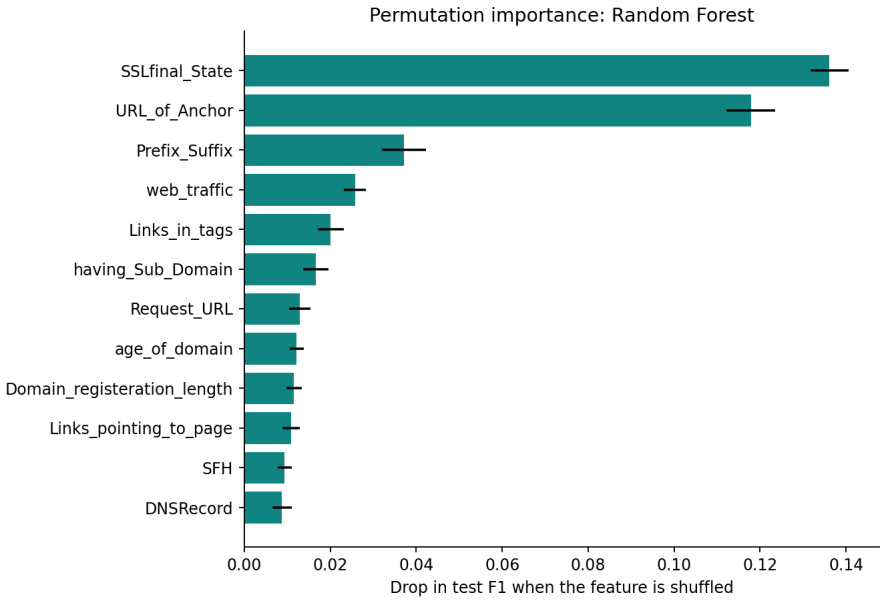



*Figure 4. Permutation importance of the Random Forest on the test set (10 repeats; explanation only, not used for selection).*

Shuffling SSLfinal_State or URL_of_Anchor reduces test F1 by 13.6 and 11.8 percentage points respectively, agreeing with the mutual-information analysis. Prefix_Suffix, web_traffic and Links_in_tags follow. Importance describes association with the model's predictions, not causation.

| Random Forest inputs | Test F1 | False alarms | Missed phishing |
|---|---|---|---|
| All 30 features | 95.91% | 38 | 42 |
| Top 10 by mutual information | 93.50% | 78 | 51 |

The top-10 subset (Prefix_Suffix, having_Sub_Domain, SSLfinal_State, Domain_registeration_length, Request_URL, URL_of_Anchor, Links_in_tags, SFH, age_of_domain, web_traffic) was selected inside each training fold. Test F1 fell by 2.40 percentage points and false alarms doubled from 38 to 78, so the weaker features still add useful signal. This extension reuses the full-model settings and is therefore exploratory rather than a fully nested selection. Feature-extraction cost was not measured.

# 9 Conclusion, limitations and future work

Within a leakage-aware, reproducible design, Random Forest was the strongest of eleven models: it was selected by cross-validation, achieved 96.38% accuracy and 95.91% phishing F1 on unseen data, and its advantage over each rival was statistically significant on this test set. It still missed 42 phishing records and produced 38 false alarms. Deeper and recurrent neural networks did not surpass tree ensembles on these tabular features, and reducing the input set to ten features lowered performance.

Limitations: the dataset is historical (donated 2015) and uses pre-extracted indicators, including reputation services whose behaviour may have changed; raw domains and timestamps are absent, so domain- or time-disjoint testing is impossible; exact-pattern grouping does not remove all collection bias; 64 conflicting patterns cap achievable accuracy; and one outer split with modest searches limits generalisation of the model ranking. The results therefore support Random Forest for this benchmark, not production readiness.

Future work: evaluate on a recent, independent dataset with domain and time separation; select an operating threshold using explicit error costs and calibrate probabilities; use repeated outer cross-

validation; and extract features live from URLs to test a browser-based detector. The same code, split manifest and predictions are retained for Assessment 4.

## 10 Presentation structure and division of work

| Slides | Presenter | Content | Time |
|---|---|---|---|
| 1–2 | A | Group information, project title, roadmap and speaker allocation | 0:45 |
| 3 | A | Introduction and motivation; research question | 0:40 |
| 4–6 | A | Dataset description, exploratory analysis, data-quality audit | 2:05 |
| 7–8 | A | Preprocessing, feature engineering, evaluation design | 1:20 |
| 9 | A → B | Algorithm and model overview; handover | 0:50 |
| 10–11 | B | Cross-validation selection; held-out test performance | 1:30 |
| 12–14 | B | ROC curves, confusion-matrix error analysis, statistical significance | 1:50 |
| 15–16 | B | Deep learning findings; explainability and feature selection | 1:25 |
| 17–18 | B | Conclusion, limitations, future work; references and questions | 1:00 |

Presenter A ≈ 5 min 40 s; Presenter B ≈ 5 min 45 s; total ≈ 11 min 25 s (limits: 6 minutes per member, 15 minutes per group). Slides 19–20 are backup material for questions.

## References


Anti-Phishing Working Group. (2026, February 18). Phishing activity trends report: 4th quarter 2025. https://docs.apwg.org/reports/apwg_trends_report_q4_2025.pdf

Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32. https://doi.org/10.1023/A:1010933404324

Dietterich, T. G. (1998). Approximate statistical tests for comparing supervised classification learning algorithms. Neural Computation, 10(7), 1895–1923. https://doi.org/10.1162/089976698300017197

Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? Advances in Neural Information Processing Systems, 35, 507–520.

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735

Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. ACM Transactions on Knowledge Discovery from Data, 6(4), Article 15. https://doi.org/10.1145/2382577.2382579

Mohammad, R., & McCluskey, L. (2012). Phishing websites [Data set]. UCI Machine Learning Repository. https://doi.org/10.24432/C51W2X

Mohammad, R. M., Thabtah, F., & McCluskey, L. (2014). Predicting phishing websites based on self-structuring neural network. Neural Computing and Applications, 25(2), 443–458. https://doi.org/10.1007/s00521-013-1490-z

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., … Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems, 32, 8024–8035.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Scikit-learn developers. (n.d.). StratifiedGroupKFold. Scikit-learn. Retrieved September 25, 2026, from https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedGroupKFold.html

## Appendix: reproducibility record

Python 3.12.3, NumPy 2.3.5, pandas 2.2.3, SciPy 1.17.0, scikit-learn 1.8.0; PyTorch 2.14.0+cu130 for the recurrent models; seed 565. Source ARFF SHA-256: 3bcf93b1612306537ee2737751f1bac185b2a77886ab376644c3bf0c7f7cb3e6. Run train.py then deep_learning_extension.py to regenerate every table and figure; the results folder retains the split manifest, audit, CV tables, per-record predictions, significance tests and environment records.

AI assistance acknowledgement: [State here, if required by your unit, how AI tools were used in preparing this work.]